Git - Version Control System

Day 2

Ionuț Ciocîrlan
Kluger Training

Remote Repositories

- Versions of your project hosted on the internet or another network
- Make projects accessible to collaborators
- Central hub for sharing changes and updates between team members

Showing Remotes

- **Command:** `git remote`
  - Shows only names of remotes
- `-v` / `--verbose` option to also show URL
- Cloned repositories have remote set up automatically

Managing Remotes

- **Add remote:** `git remote add <name> <url>`
- **Remove remote:** `git remote remove <name>`
- **Rename remote:** `git remote rename <old-name> <new-name>`
- **Change remote URL:** `git remote set-url <remote> <url>`

Practice: Managing Remotes

1. Create a GitHub account if you don't already have one.
2. Set SSH keys for your account (short guide).
3. Create a new public repository in your account. Name it `new_project`.
4. Connect your local repository in `new_project` to this remote repository by adding it as a remote.

Fetching Changes

- Fetching allows you to download changes from a remote repository without merging them into your local branch.
- **Fetch all changes:** `git fetch <remote>`
- **Fetch single branch:** `git fetch <remote> <branch>`
- **View commit history on remote:** `git log <remote>/<branch>`
- **View diff between local and remote:**

```
git diff <branch> <remote>/<branch>
git switch <local-branch>
git merge <remote>/<branch>
```

Practice: Fetching Changes

1. Push current `main` / `master` branch to remote (`git push origin <branch>`).
2. Create a commit in `new_project` on `github.com`.
3. Fetch remote changes locally.
4. View commit history on `origin/main`.
5. View changes in `origin/main` that are not yet in (local) `main`.
6. Merge `origin/main` into your local `main`.

Pulling Changes

- Pulling combines fetching and merging in one step.
- **Command:** `git pull <remote> <branch>`

Practice: Pulling Changes

1. In `git-playground` make sure you're on the `main` branch. View commit history.
2. Pull `origin/main` from local `main`. View commit history again.

Pushing Changes

- Pushing sends your local changes to a remote repository.
- **Command:** `git push <remote> <branch>`
- **Push new branch:** `git push -u <remote> <branch>`
- **Check tracked remote branch:** `git branch -vv`

Practice: Pushing Changes

1. Pull latest changes in `git-playground`.
2. Pick an issue from here: git-playground Issues.
3. Create a new branch locally from `main` for that issue.
4. Make the necessary changes and commit on the new branch. Multiple commits are also fine.
5. Push local branch to remote repository.

Tagging

- Mark specific points in your project's history, such as release versions.
- **Lightweight tag:**
  - Simple pointer to a specific commit
  - `git tag <tagname>`
- **Annotated tags:**
  - Store additional metadata (author, date, message)
  - `git tag -a <tagname> -m "<message>"`

Tagging (Continued)

- **Tagging a commit different than the latest one:**

```
git tag <tagname> <commit-hash>
```

- **List all tags:**

```
git tag
```

Pushing Tags

- **Pushing a single tag:** `git push <remote> <tagname>`
- **Pushing all tags:** `git push --tags`

Deleting Tags

- **Local tags:** `git tag -d <tagname>`
- **Remote tags:** `git push <remote> --delete <tagname>`

Checking Out Tags

- Although tags are not branches, you can check out a specific tag to view the project state at that point in time.
- This places your repository in a "detached HEAD" state.
- **Command:** `git checkout <tagname>`

Practice: Tagging

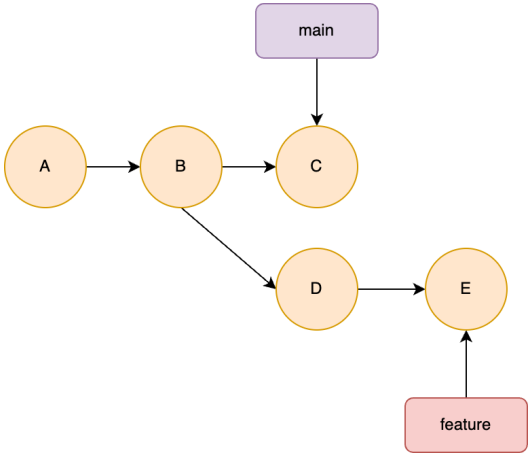
1. Create a simple tag pointing to your latest commit in `new_project`.
2. Create an annotated tag pointing to an older commit in `new_project`.
3. List all tags.
4. Push tags to remote repo. Go to your project on `github.com` to see the tags.
5. Delete tag to latest commit both locally and remote.
6. Checkout to annotated tag.

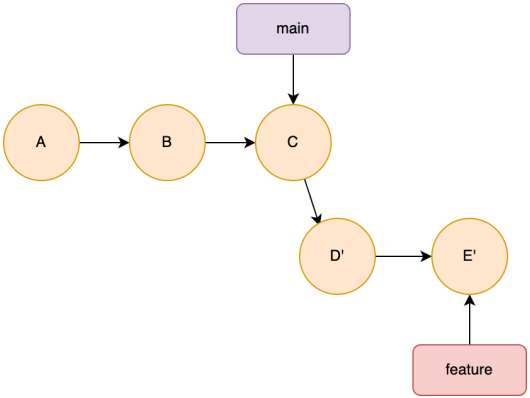
Rebasing

- Rebasing can be used as an alternative to merging.
- Creates linear commit history without merge commits.
- Rewrites history.
- **Rebasing `feature` onto `main`:**

```
git switch feature
git rebase main
```

Rebasing Illustrated





Practice: Rebasing

1. Go to your feature branch (the branch you created for your issue) in `git-playground`. Create a new branch `experiment-rebase`.
2. Move to `main` branch. Get latest changes from remote `main`.
3. Rebase your `experiment-rebase` branch onto `main`.
4. Go to your feature branch (the branch you created for your issue) in `git-playground`. Create a new branch `experiment-merge`.
5. Move to `experiment-merge` branch and merge `main` into it.

Interactive Rebasing

- Allows to edit, squash, reorder, or remove commits.
- **Command:** `git rebase -i <branch>`
- **Available actions:**
  - `pick`: Keep the commit as-is.
  - `reword`: Edit the commit message.
  - `edit`: Amend the commit's changes or message.
  - `squash`: Combine this commit with the previous one.
  - `drop`: Remove the commit.

Practice: Interactive Rebasing

1. In your `new_project` project, create a new branch `rebase-experiments`.
2. Switch to this branch and do some interactive rebases:
   a. Drop a commit
   b. Change message for an older commit
   c. Split a commit
   d. Squash three commits into one
3. Push your branch to the remote repo.

Stashing

- Save changes and revert working directory to a clean state.
- Useful to stash away work in progress.
- **Stash changes:** `git stash`
- **Stash under a name:** `git stash save "stash name"`
- **Show stash stack:** `git stash list`

Applying and Dropping Stashes

- **Apply most recent stash:** `git stash apply`
- **Apply specific stash:** `git stash apply stash@{index}`
- **Drop specific stash:** `git stash drop stash@{index}`
- **Drop all stashes:** `git stash clear`
- **Apply and remove stash in one step:** `git stash pop`

Practice: Stashing

1. Create four stashes. Try out both simple and named stashes. View stashes.
2. Apply second stash. Commit the change.
3. Remove applied stash. View stashes.
4. Apply and remove latest stash. Commit the change. View stashes.
5. Drop all stashes.

Cleaning the Working Directory

- Removing untracked directories/files no longer needed.
- **Preview the files that will be removed:** `git clean -n`
- **Remove untracked files:** `git clean -f`
- **Remove untracked directories:** `git clean -fd`
- **Clean ignored files:** `git clean -fx`

The .gitignore File

- Preventing unnecessary files from being tracked.
- Uses glob syntax.
- Example `.gitignore` content:

```
*.pyc
.idea
**/*.mo
```

- **See if a file is ignored:** `git check-ignore -v <file>`

Practice: .gitignore

1. Create a directory `logs` with two files in it: `app.log` and `app.1.log`.
2. Check `git status`.
3. Create a `.gitignore` file and add a rule to it that all files inside `logs` directory are ignored.
4. Check `git status` again.
5. Add a file `app.txt` inside `logs`. Is it also ignored?
6. Change rule in `.gitignore` so that only `.log` files in `logs` are ignored.
7. Check `git status` again.